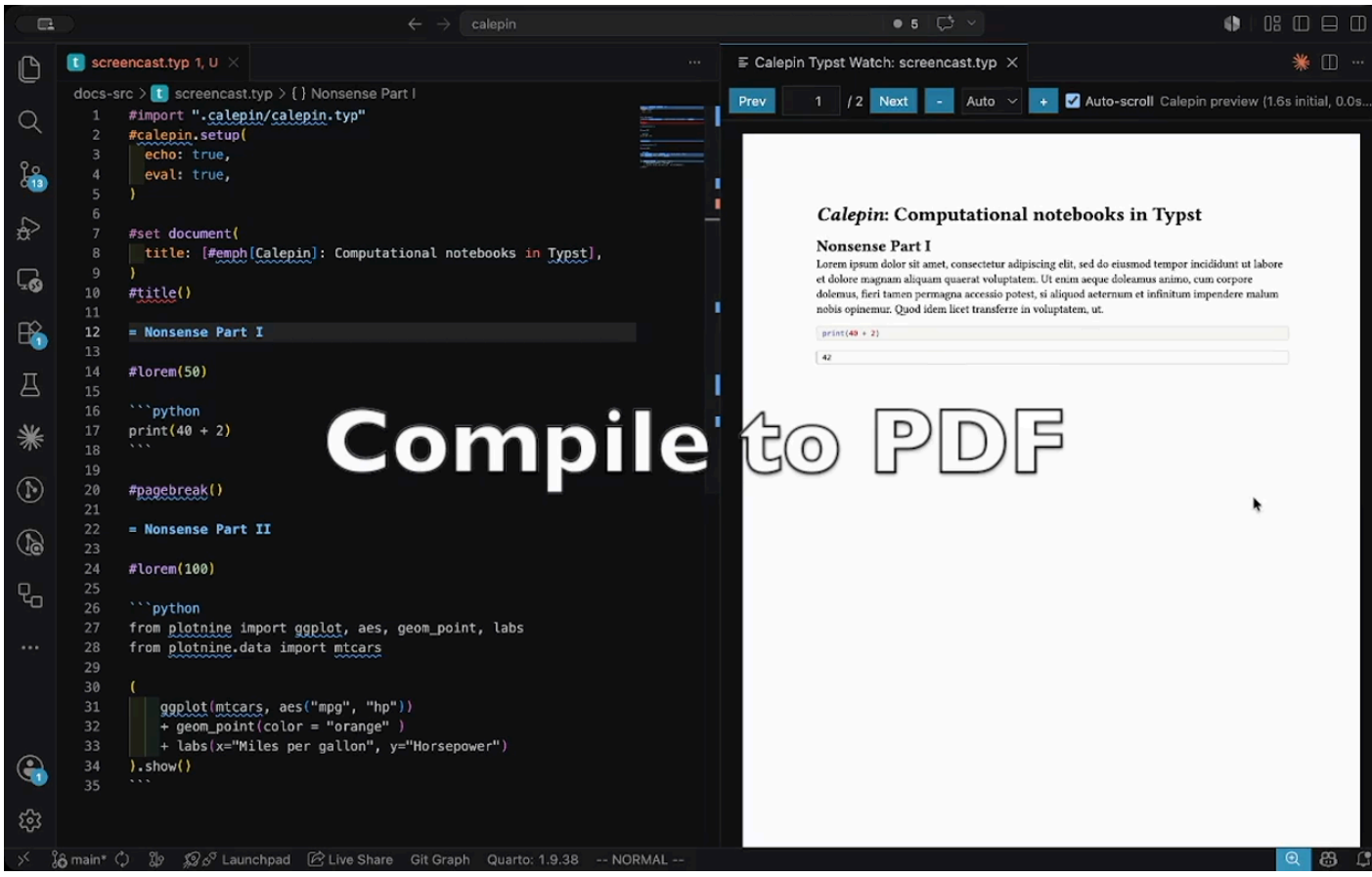
Editor integration

Calepin documents are ordinary Typst files, so you can keep your editor's language tooling and preview while Calepin evaluates computational chunks.

The Tinymist extension provides Typst language tooling and live document preview in VS Code.

The *Calepin for Typst* extension adds start and stop commands for Calepin in VS Code, Cursor, Positron, and other VSX-compatible editors. Install it from the Visual Studio Marketplace for VS Code, or from Open VSX for Cursor, Positron, VSCodium, and other editors that use the Open VSX registry. This short screencast shows the workflow alongside Tinymist preview.



Video: /assets/calepin_vscode.mp4

Workflow

Tinymist and Calepin do different jobs, and both must be running. Tinymist ships its own Typst binary and renders the preview; it does not know what a code chunk is and never executes one. Calepin executes the chunks and stores their results on disk. The preview then picks up those stored results like any other file the document reads.

1. Open the notebook in VS Code, Cursor, or Positron.
2. Run **Typst: Calepin** from the command palette. This starts `calepin watch <file> --eval-only` in the background, which evaluates chunks but leaves rendering to your editor.
3. Run **Typst Preview: Preview Opened File** from Tinymist.
4. Edit and save. Calepin re-evaluates the chunks whose code changed and rewrites the results; Tinymist re-renders and the preview updates.

Run **Typst: Stop Calepin** to stop the background watcher.

The document show rule

A notebook previewed this way must apply the document show rule:

```
#import "/.calepin/calepin.typ" as calepin
#show: calepin.document
```

This rule is what replaces executable code fences with their stored results. When Calepin drives Typst itself, it supplies the rule internally, so `calepin compile` produces a correct PDF or HTML file whether or not the line is present. Tinymist compiles the notebook directly, with no such wrapper, so without the line nothing rewrites the fences.

> **Note**
>
> The symptom is specific: the preview shows your prose and your code, but no chunk output, while the file produced by `calepin compile` looks correct. Inline results such as `#py[...]` still appear, because those are ordinary function calls that read the stored results directly and do not depend on the show rule. If block chunks are blank and inline results are not, the show rule is missing.

`calepin new paper.typ` writes the line into the notebook it creates, so new documents have it already.

Configuration

The extension runs `calepin watch <file> --eval-only` without a `--config` argument, and Calepin does not auto-discover configuration files. A `calepin.toml` that sets interpreter paths under `[executables]` therefore has no effect on chunks evaluated through the extension, and an engine that is not on your `PATH` will not run.

Until the extension grows a setting for this, either make the interpreters discoverable on `PATH`, or skip the extension command and start the watcher yourself in a terminal:

```
calepin watch --config calepin.toml paper.typ --eval-only
```

Tinymist preview works the same way against a watcher started in a terminal. The only setting the extension exposes today is `calepin.binaryPath`, which selects the `calepin` binary to run.